

MBA IN **SOFTWARE ENGINEERING**

Software Development Paradigms

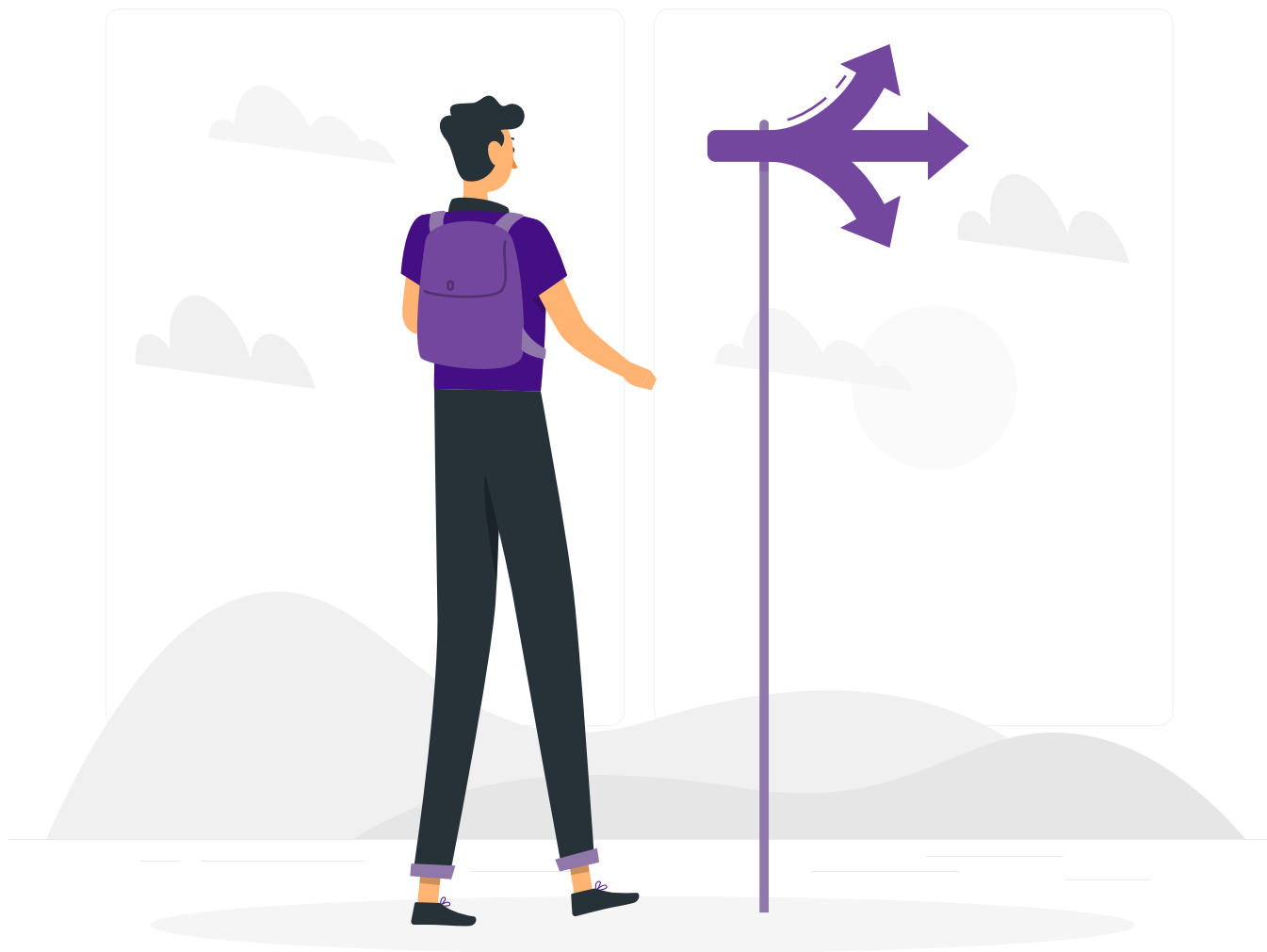
Prof. Helder Prado Santos

The trustworthiness, originality,
and legality of the educational content presented is responsibility of the professor.

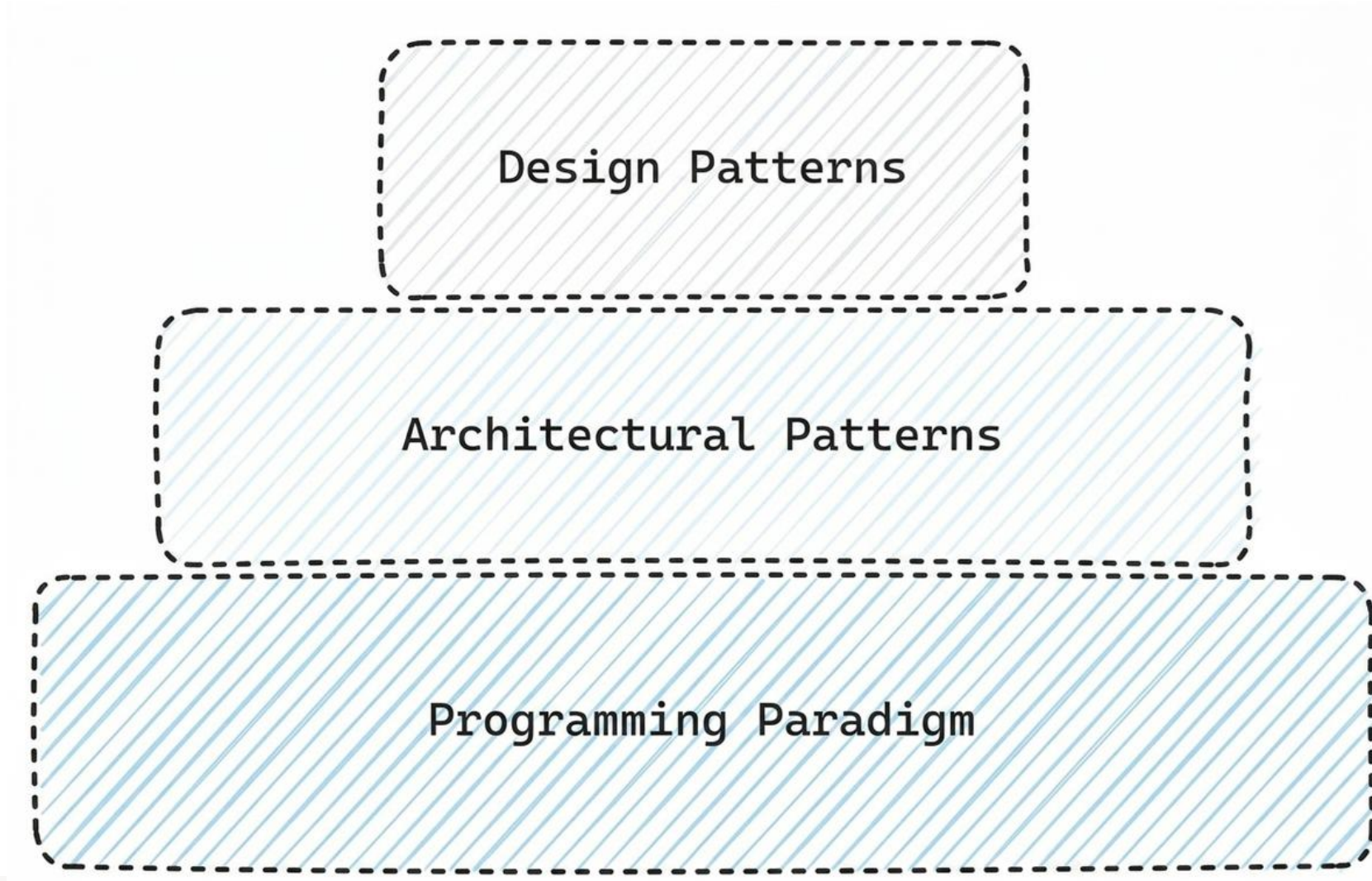
The total or partial reproduction of this material without authorization is prohibited.

Law No.
9610/98

Programming Paradigm



The Paradigm is the Basis of the Solution



Imperative Paradigm (1950s)

- **Context:** The imperative paradigm was one of the first programming styles adopted, at a time when the focus was on directly controlling computer resources.
- **Initial Languages:** Assembly, Fortran.
- **Characteristics:** Sequential commands, use of variables to store state, and explicit control of execution flow.
- **Impact:** This paradigm served as the basis for structured programming and offers direct control over hardware.

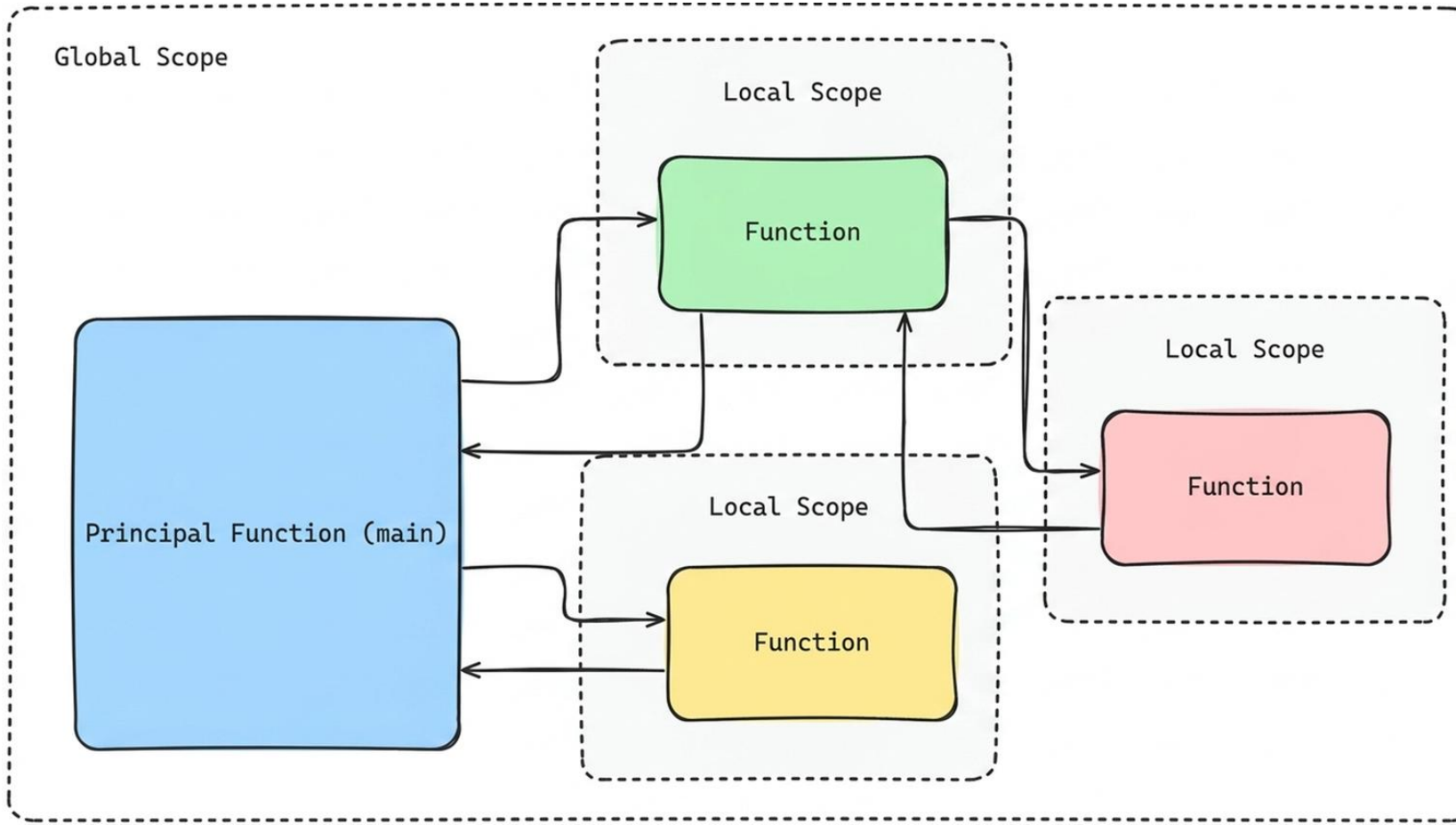
Functional Paradigm (1950s)

- **Context:** Based on the mathematical theory of functions, this paradigm grew with the popularization of specific languages.
- **Languages:** Lisp, Haskell.
- **Characteristics:** The functional paradigm avoids mutable states and side effects, prioritizing pure functions and immutability.
- **Impact:** It offers advantages in concurrent programming and data processing, being widely used in mathematical and scientific computing problems, as well as in modern frameworks for data manipulation and concurrency.

Procedural Paradigm (1960s)

- **Context:** The procedural paradigm emerged as a natural evolution of the imperative, by introducing structures that organize code into procedures or subroutines. This helped make the code more modular and reusable.
- **Languages:** C, Fortran, COBOL, Pascal.
- **Characteristics:** Organization of code into procedures or functions that can be called to perform specific tasks. The program is built as a sequence of calls to these procedures, which facilitates controlled execution flow.
- **Advantages:** Better modularity compared to pure imperative, allowing code reuse and reduction of redundancy.
- **Examples of use:** Scientific applications, engineering, and business systems that require repetitive and organized processes.

Procedural Paradigm (1960s)



Structured Paradigm (1960s)

- **Context:** As programs became more complex, the need arose to create a more organized structure for the code.
- **Languages:** C, Pascal.
- **Characteristics:** Introduction of control structures (such as loops and conditionals), subroutines, and functions, facilitating code maintenance and readability.
- **Impact:** The structured paradigm paved the way for the development of more organized programs, reducing "spaghetti code" (disorganized and difficult-to-read code).

Structured Paradigm (1960s)

Code

```
goto label_1;
```

```
statement_1;  
statement_2;
```

```
label_1:
```

```
statement_3;  
statement_4;
```

```
goto label_3;
```

```
statement_5
```

```
label_2;
```

Code

```
goto label_4;
```

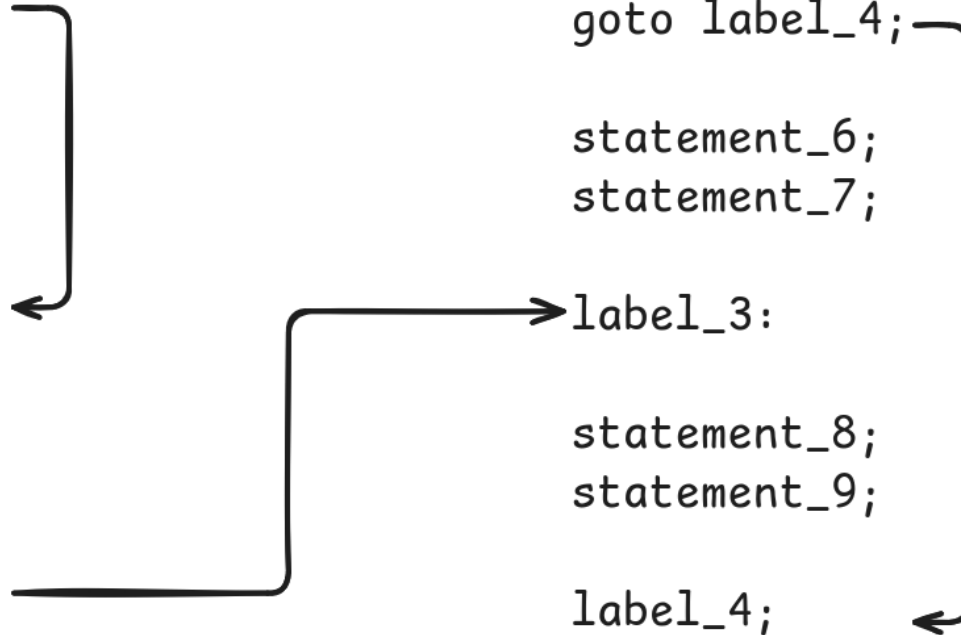
```
statement_6;  
statement_7;
```

```
label_3:
```

```
statement_8;  
statement_9;
```

```
label_4;
```

```
statement_10;  
statement_11;
```



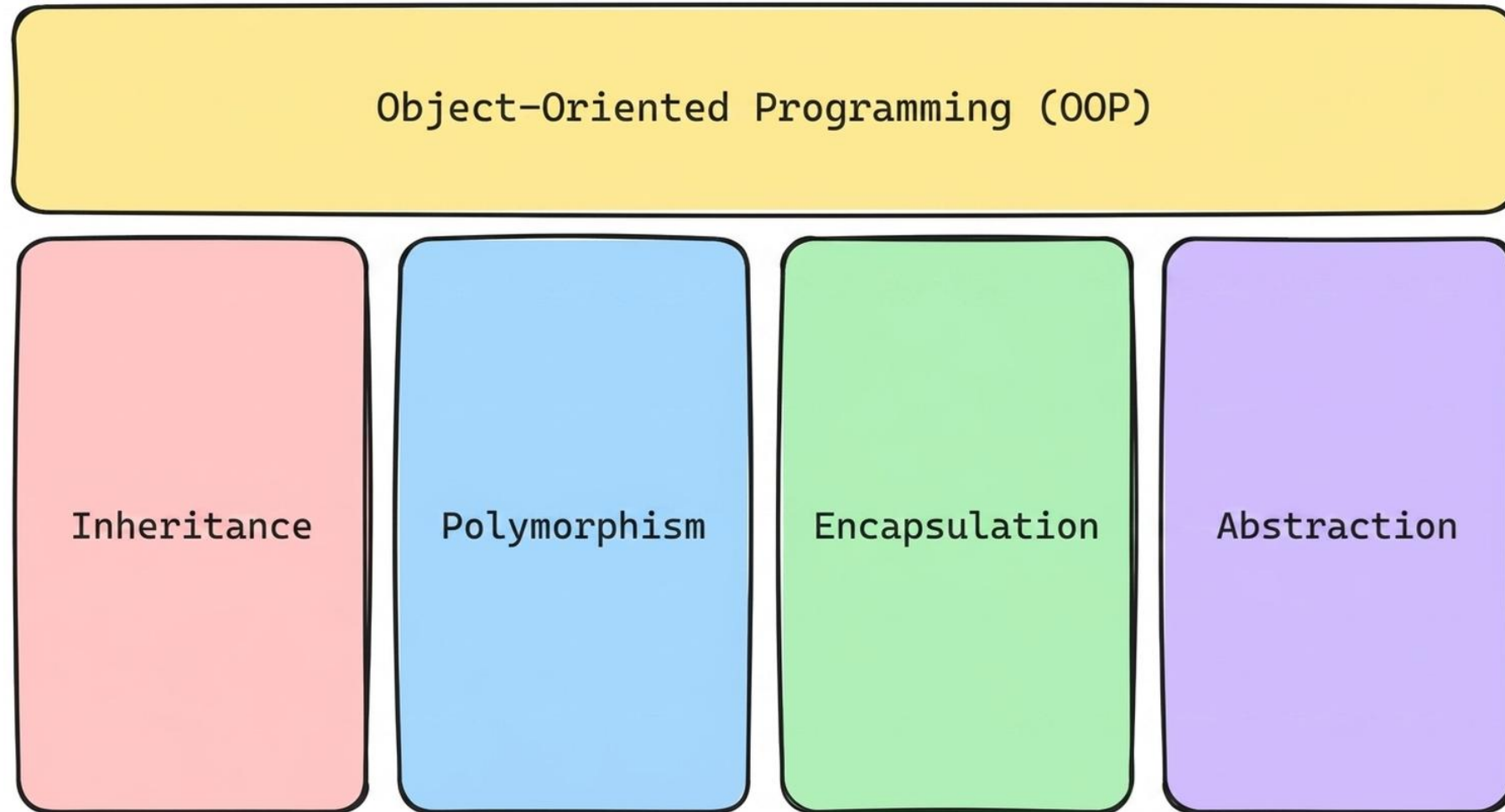
Logic Paradigm (1970s and 1980s)

- **Context:** Created for artificial intelligence applications and logic-based problem solving.
- **Languages:** Prolog, Datalog.
- **Characteristics:** Focuses on declaring relationships and rules that describe the problem, leaving it to the computer to find logical solutions.
- **Impact:** Although not widely used for commercial applications, it is essential in AI areas such as natural languages and logical inference.

Object-Oriented Paradigm (OO) (1980s)

- **Context:** Emerged in response to the need for modularity and code reuse for increasingly complex systems.
- **Languages:** Java, C_
- **Characteristics:** Organizes code into "objects" that represent real-world or conceptual entities, encapsulating data and behaviors.
- **Impact:** It became the dominant paradigm in corporate development and games, promoting the creation of reusable libraries and helping to manage the complexity of large systems.

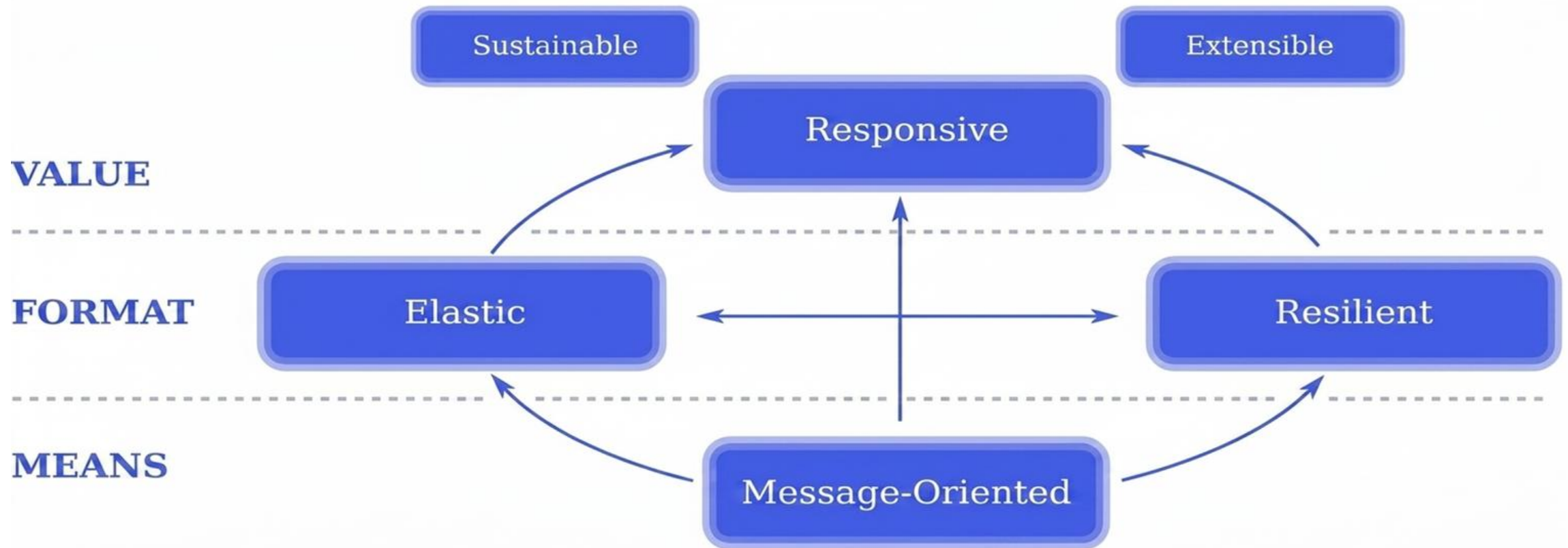
Object-Oriented Paradigm (OO) (1980s)



Reactive Paradigm (2000 onwards)

- **Context:** With the increasing demand for real-time and responsive systems, the reactive paradigm became more relevant.
- **Tools and Technologies:** RxJS, Reactor, reactive programming frameworks.
- **Characteristics:** Uses asynchronous data streams and event-driven programming, where changes in one part of the system automatically react to changes in another.
- **Impact:** Essential for modern applications such as real-time dashboards, streaming platforms, and distributed systems.

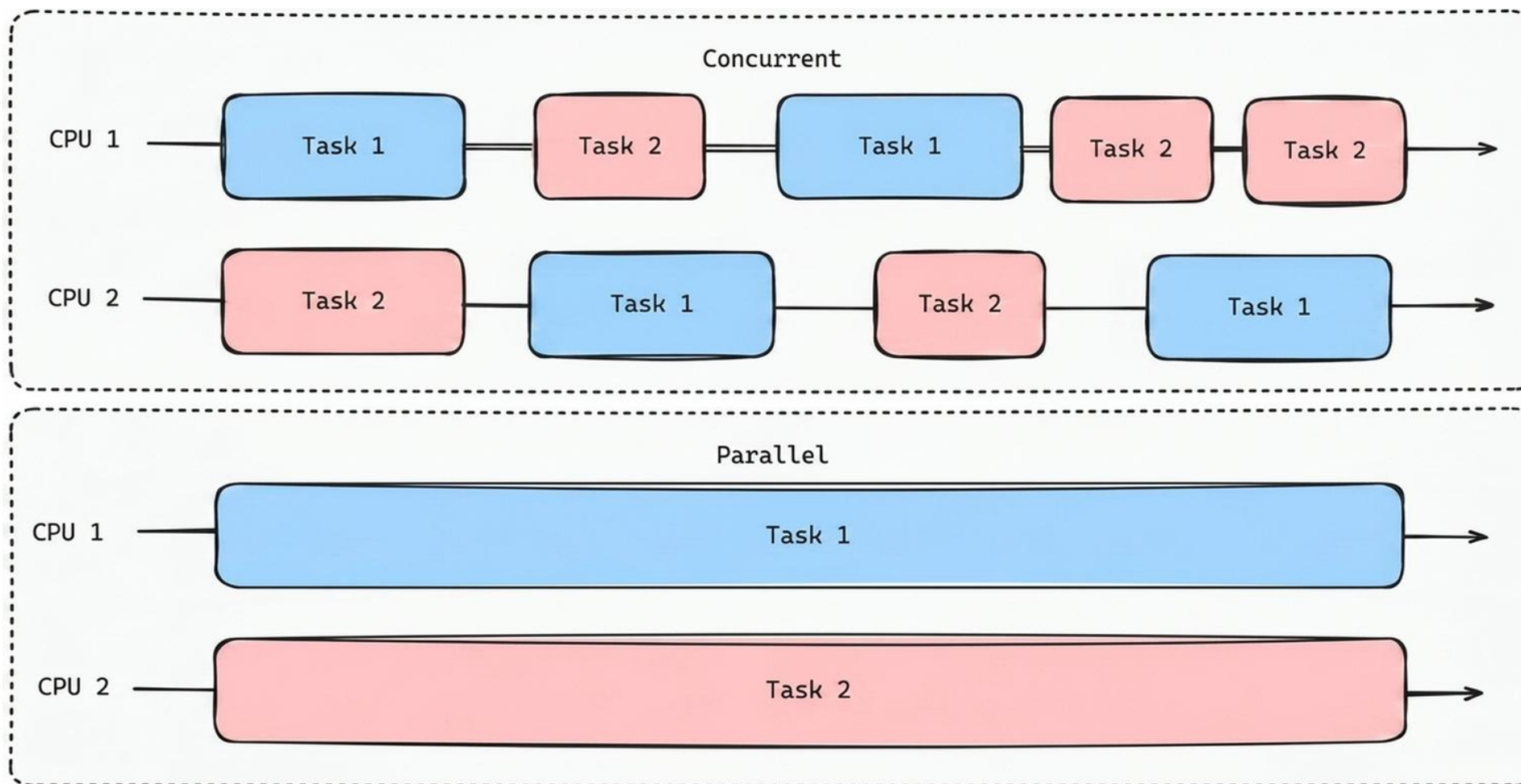
Reactive Paradigm (2000 onwards)



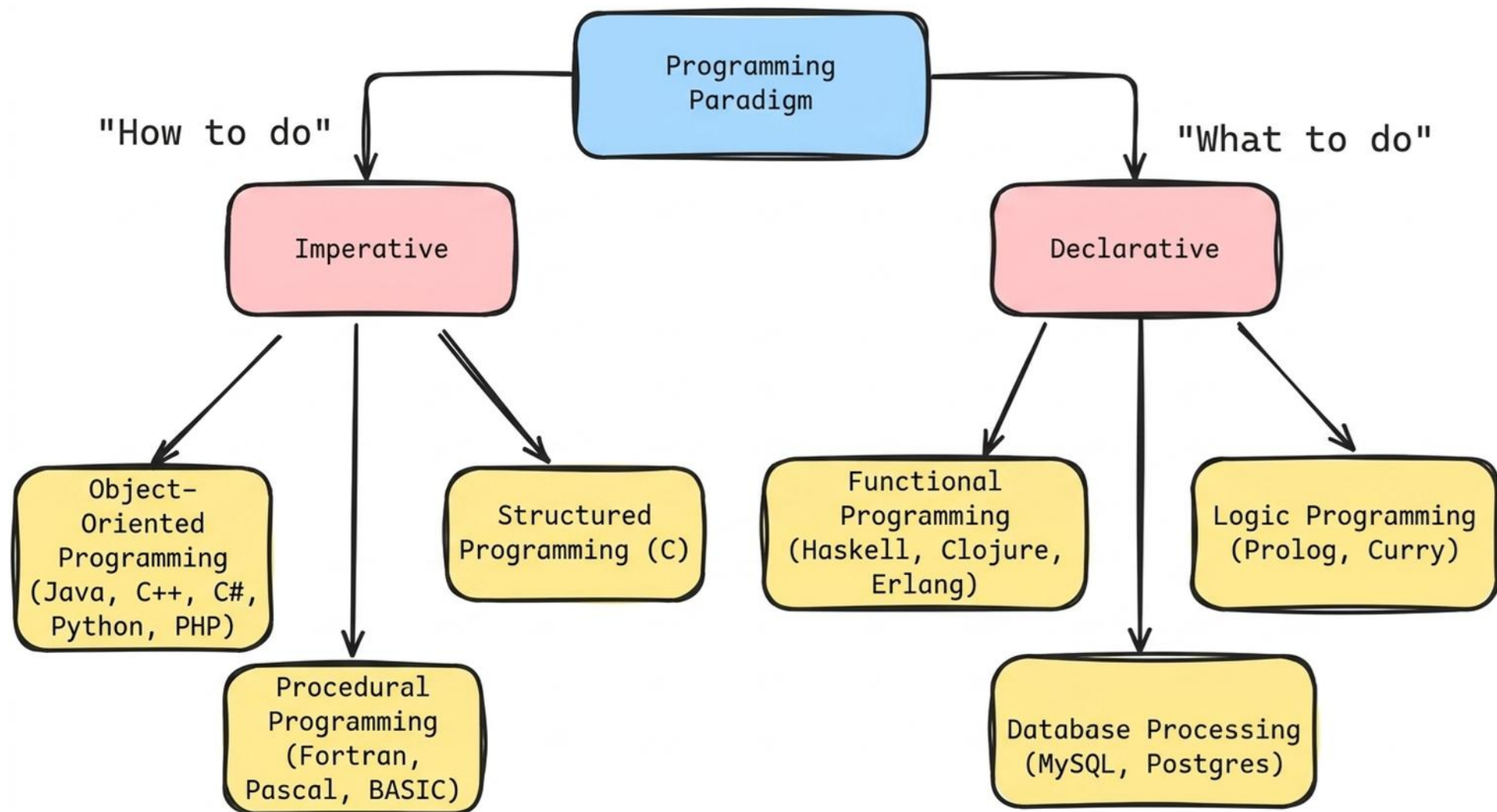
Concurrent and Parallel Paradigms (throughout the decades)

- **Context:** With the advent of multicore processors, concurrent and parallel programming became necessary to better utilize hardware.
- **Tools and Technologies:** Threads, OpenMP, MPI, actor frameworks like Akka.
- **Characteristics:** Focus on simultaneous execution of multiple tasks to increase the performance and efficiency of applications that process large volumes of data.
- **Impact:** Essential for areas such as scientific computing, simulations, and big data, helping to create faster programs capable of processing large volumes of data in parallel.

Concurrent and Parallel Paradigms (throughout the decades)



Imperative and Declarative Paradigm



Imperative and Declarative Paradigm

Imperative Paradigm	Declarative Paradigm
How to do	What to do
More verbose	Less verbose
Mutable variables	Immutability
Sequential commands	Set of statements
More explicit	Less explicit
Greater focus on data flow	Greater focus on logic
More readable and alterable	Harder to understand
Harder to scale	Easier to scale

Making a cup of coffee using the paradigms

Imperative:

- "John, get the coffee maker. Add water up to the indicated level. Open the coffee grounds compartment. Put in three scoops of grounds. Close the compartment. Place the coffee maker on the base. Press the power button. Wait until the coffee is ready. Take a cup and pour the coffee into it."

Declarative:

- "John, make a cup of coffee."

Practical example of the paradigms



```
# List of numbers
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Variable to store the sum of even numbers
soma_pares = 0

# Loop to iterate through the list and sum the even numbers
for numero in numeros:
    if numero % 2 == 0: # Checks if the number is even
        soma_pares += numero # Adds the even number to the sum

print("Sum of even numbers:", soma_pares)
```

Practical example of the paradigms



```
# List of numbers
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Sum of even numbers using higher-order functions
soma_pares = sum(filter(lambda x: x % 2 == 0, numeros))

print("Sum of even numbers:", soma_pares)
```

Practical example of the paradigms



```
SELECT name, age  
FROM customers  
WHERE city = 'São Paulo' AND age > 30;
```

Practical example of the paradigms

```
class Livro:
    def __init__(self, titulo, autor, ano_publicacao):
        self.titulo = titulo
        self.autor = autor
        self.ano_publicacao = ano_publicacao
        self.disponivel = True

    def emprestar(self):
        if self.disponivel:
            self.disponivel = False
            print(f'O livro "{self.titulo}" foi emprestado.')

        self.disponivel = True
        print(f'O livro "{self.titulo}" foi devolvido.')
```

Practical example of the paradigms

```
class Biblioteca:
    def __init__(self):
        self.livros = []

    def adicionar_livro(self, livro):
        self.livros.append(livro)
        print(f'0 livro "{livro.titulo}" foi adicionado à biblioteca.')

    def listar_livros(self):
        print("\nLista de livros na biblioteca:")
        for livro in self.livros:
            print(livro)

    def emprestar_livro(self, titulo):
        for livro in self.livros:
            if livro.titulo == titulo:
                livro.emprestar()
                return
        print(f'0 livro "{titulo}" não foi encontrado na biblioteca.')

    def devolver_livro(self, titulo):
        for livro in self.livros:
            if livro.titulo == titulo:
                livro.devolver()
                return
```

Practical example of the paradigms



```
# Creating some books
livro1 = Livro("1984", "George Orwell", 1949)
livro2 = Livro("O Senhor dos Anéis", "J.R.R. Tolkien", 1954)
livro3 = Livro("O Código Da Vinci", "Dan Brown", 2003)

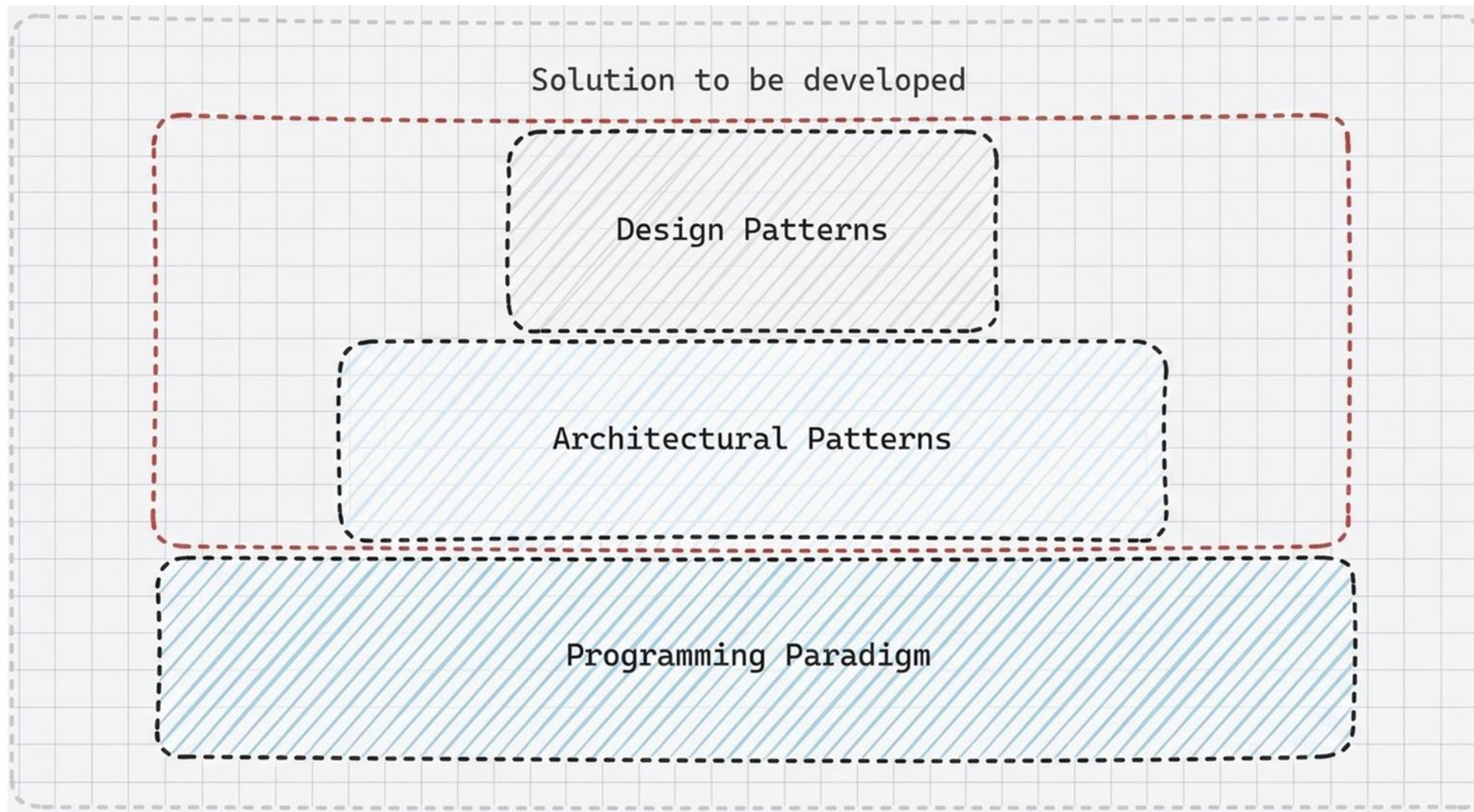
# Adding books to the library
biblioteca.adicionar_livro(livro1)
biblioteca.adicionar_livro(livro2)
biblioteca.adicionar_livro(livro3)

# Listing books in the library
biblioteca.listar_livros() # [livro1, livro2, livro3]

# Lending a book
biblioteca.emprestar_livro("1984")
biblioteca.listar_livros() # [livro2, livro3]

# Returning a book
biblioteca.devolver_livro("1984")
biblioteca.listar_livros() # [livro1, livro2, livro3]
```

Choosing the Appropriate Paradigm



Thank you!

Prof. Helder Prado Santos | [linkedin.com/in/helderprado](https://www.linkedin.com/in/helderprado)

MBA **USP**
ESALQ